

AI Logistics Autopilot

An AI-powered logistics command center that automates vehicle assignment, route optimization, delay triage, disruption response and fleet utilization — transparent, explainable and honestly labelled. This document covers the complete architecture, data model, API reference, AI engines, live data sources, setup and deployment.

Complete Technical Documentation

Two-service edition — frontend & backend services, version 1.0

Table of Contents

1 • Project Overview	2
2 • Technology Stack	2
3 • Architecture & Data Pipeline	3
4 • AI Models & Engines	4
5 • Real Data Sources	5
6 • Features & Real-World Scenarios	5
Command Center — the 09:00 stand-up	6
Shipments & AI assignment — “which truck takes this load?”	6
Route Intelligence — planning around reality	6
Consolidation — filling half-empty trucks	6
Return Loads — no truck runs empty	6
Disruption Control — the playbook button	7
Analytics, Copilot & Thresholds	7
7 • API Reference (31 endpoints)	7
8 • Security & Data Honesty	8
9 • Running the Project	8
10 • SIH Innovation Register & Honest Limitations	9

1 • Project Overview

AI Logistics Autopilot is an AI-powered freight command centre for Indian road logistics, built for Smart India Hackathon 2026. It gives a logistics operator one screen where every consignment, truck and driver is live: the system plans routes with real road data, scores delay risk before SLA breach, assigns the best vehicle automatically, consolidates part-loads, fills empty return legs and walks dispatchers through disruptions with a click-through playbook.

31	7 + 1	22	45
REST endpoints (FastAPI)	AI engines + LLM copilot	Indian hubs · 15 corridors	Governed AI thresholds

No hallucinated data. Every AI output carries an honest data-status label — **live** (real external feed), **rule_based / optimization** (deterministic, explainable engines) or **demo** (labelled sample fleet data). No number is presented as real when it is not.

The product targets the daily decisions a dispatch floor makes under time pressure: which truck takes which load, which route survives tomorrow's weather, which shipments can share a trailer, and what to do the moment a corridor floods. Each decision is scored, explained factor-by-factor, and logged — the operator stays in control while the autopilot does the arithmetic.

2 • Technology Stack

The project is a two-service stack: a TypeScript frontend (UI + transparent API proxy) and a Python FastAPI backend that owns every business rule. There is no third API layer — **FastAPI is the only API** in the system.

Layer	Technology	Role
Frontend	Next.js 16 (App Router), React 19, TypeScript 5, Tailwind CSS 4 + shadcn/ui, Framer Motion, Recharts, TanStack Query/Table, Zustand	Marketing site, sign-in/sign-up, command center and 10 operational views. Proxies every /api/* call to the backend (next.config.ts rewrites) — one origin, no CORS.
Backend	Python 3.12, FastAPI, Pydantic v2, Uvicorn, SQLAlchemy 2.x, PyYAML, httpx	All 31 REST endpoints, the 7 AI engines, session auth (scrypt), org-scoped multi-tenancy, live-data clients, thresholds governance and audit trail. Interactive OpenAPI docs at /docs.

Layer	Technology	Role
Database	SQLite (file) via SQLAlchemy; schema auto-created and self-seeded on first boot; PostgreSQL-ready models	13 models: Organization, User, Session, Vehicle, Driver, Shipment, ShipmentEvent, Recommendation, Disruption, ThresholdConfig, Notification, DemoRequest, RoadLeg.
External APIs	Open-Meteo (weather) · BRouter + OSRM over OpenStreetMap (road routing) · z-ai LLM bridge for the copilot	Keyless and free; every value fetched from these is labelled live in the UI with source attribution; cached and gracefully degraded when offline.

Fast answers. Which API? One REST API, served by FastAPI under `/api/*` with a unified error envelope (`{error:{code,message,details,requestId}}`), cookie sessions (scrypt-hashed passwords, 7-day TTL) and role-based access (ADMIN / OPS_MANAGER / DISPATCHER / ANALYST). **Which model?** Seven deterministic, explainable scoring engines whose weights are governed by a versioned YAML config, plus a large language model (GLM, via the z-ai bridge) used only for the natural-language copilot — the LLM reads your org data but never invents numbers.

3 · Architecture & Data Pipeline

A request flows through exactly three hops. The browser talks only to the Next.js origin; Next.js rewrites `/api/*` to the FastAPI service; FastAPI answers from its engines, the database and cached live feeds.

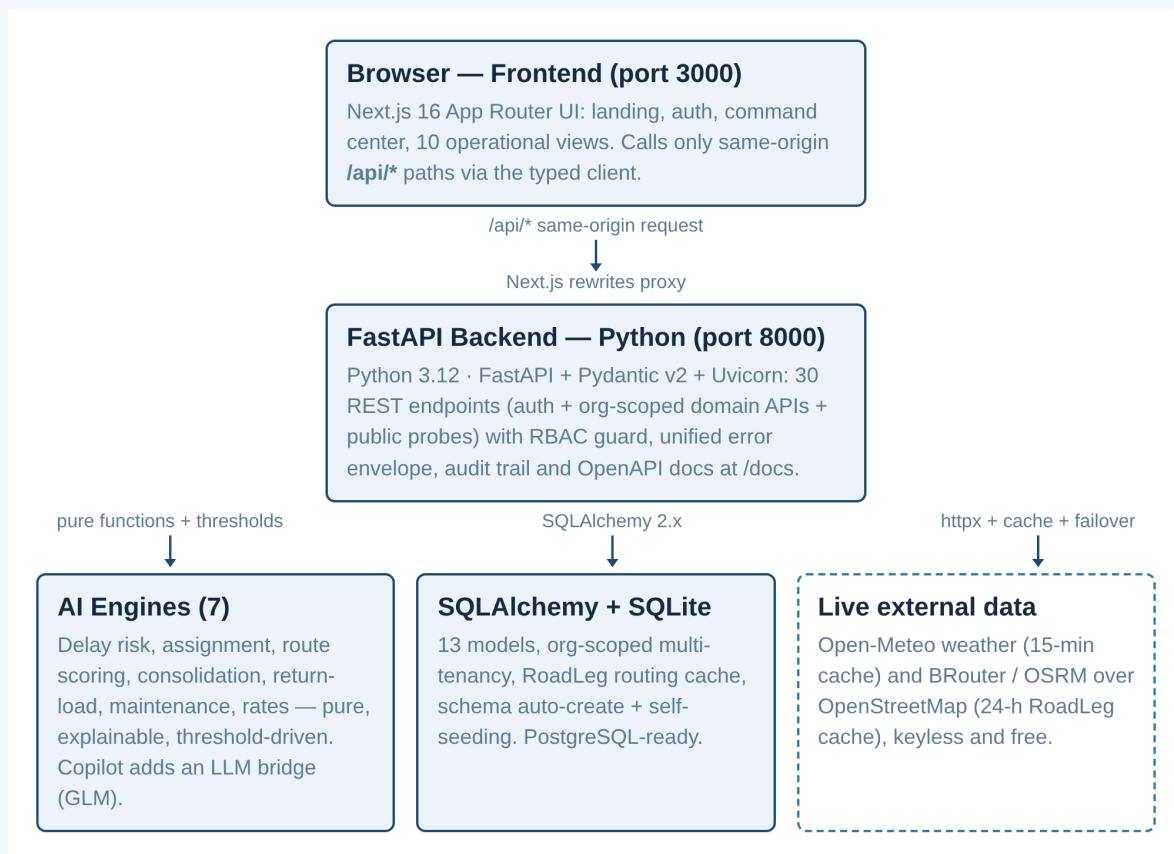


Figure 1 — Two-service architecture with the FastAPI contract boundary

- **1 • Browser → Next.js (port 3000):** the typed API client (lib/api.ts) issues same-origin fetches with TanStack Query caching, polling and optimistic invalidation.
- **2 • Next.js → FastAPI (port 8000):** next.config.ts rewrites forward /api/* transparently; in dev the Python service is auto-spawned by the Next.js instrumentation hook, so one command boots the whole stack.
- **3 • FastAPI internals:** routers validate with Pydantic → org-scoped SQLAlchemy queries → AI engines compute scores from threshold weights → live clients enrich with cached weather and road data → JSON responses are normalised to the exact frontend contract (camelCase, JS numbers).
- **Write pipeline (e.g. creating a shipment):** validate payload → resolve real road distance and duration (BRouter/OSRM, cached in RoadLeg) → fetch live weather for both ends → price the trip (fuel + tolls + driver) → score delay risk → persist with an event-log entry → return the shipment DTO plus a sources[] manifest naming every data origin used.

4 • AI Models & Engines

All decision support comes from seven deterministic engines — pure functions of your data and your governed thresholds. They are fast, reproducible and explainable: every score ships with a factor breakdown, so the UI can show *why* in one line per factor.

Engine	Input	What it computes	Label
Delay risk	Shipment, corridor, live weather, slack	0-100 delay score + low/medium/high/critical band; factors: live weather, schedule slack, corridor congestion, priority, driver hours	rule_based
Vehicle assignment	Load profile, fleet state	Ranked candidate vehicles with capacity fit, proximity, health, driver hours and priority match factors	rule_based
Route scoring	Corridor legs, fuel/toll rates, weather	3 alternatives (Fastest / Balanced / Economy) with distance, ETA, cost, tolls, fuel, CO ₂ and a risk score each	optimization
Consolidation	Unassigned loads sharing a corridor + window	Groups that fill one truck (weight + volume fill %) with computed ₹ savings vs separate trips	rule_based
Return-load match	Vehicles finishing a trip, open loads near destination	Detour km, profit estimate and a weighted match score per vehicle-load pair	rule_based
Maintenance watch	Odometer, last service, health score	Warning / critical flags with the exact rule that triggered them	rule_based
Rates & pricing	Distance, toll table, fuel price, vehicle efficiency	Trip cost split into fuel / tolls / driver, ₹ per km, CO ₂ estimate	rule_based

The 8th model. Copilot LLM: the natural-language assistant answers from your org context (at-risk shipments, fleet state, live corridor weather). It runs through a pluggable bridge — the z-ai CLI/SDK (GLM) in this environment, or any OpenAI-compatible endpoint via LLM_BASE_URL + LLM_API_KEY. If no LLM is reachable it falls back to the deterministic answer engine and labels the reply **rule_based** instead of **llm**. Answers never contain numbers the engines did not produce.

5 • Real Data Sources

External feeds are keyless, free and honestly labelled. A read-through cache (RoadLeg table for roads, in-memory TTL for weather) keeps public services hit at most once per query per day, and every endpoint degrades gracefully — if a feed is down, outputs fall back to clearly-labelled estimates rather than failing or inventing values.

Source	Provides	Caching & fallback	Attribution
Open-Meteo (api.open-meteo.com)	Current temperature, precipitation, wind, WMO code + today's rain total per hub	15-min TTL, 5-min negative cache, 7 s timeout; on failure falls back to a seasonal corridor profile labelled as estimate	shown as “Live weather · Open-Meteo”
BRouter (car-fast) with OSRM failover	Real road distance and duration for every hub pair	Read-through RoadLeg table (24 h+); retries then straight-line haversine estimate with an “estimated” label	shown as “Real road data · BRouter/OSRM (OpenStreetMap)”
OpenStreetMap	Underlying road network for the routers	served via BRouter/OSRM	OSM contributors (ODbL)
z-ai LLM (GLM)	Copilot natural-language answers grounded in org data	n/a — stateless; deterministic fallback	labelled “LLM + your data”

Fleet operations themselves (vehicles, drivers, shipment history) are a **labelled demo dataset** seeded deterministically on first boot — no GPS telemetry exists for a demo fleet, so the UI tags it “Demo data” rather than claiming live telemetry. Weather and road figures in every screenshot are genuinely live.

6 • Features & Real-World Scenarios

Each feature below maps to one screen and one real operating situation.

Command Center — the 09:00 stand-up

How it works: six KPIs (active shipments, on-time %, at-risk, utilisation, ₹/km, CO₂ today), a live-conditions strip for eight real hubs, an animated network map of trucks on corridors, a delay radar and an approve/reject recommendation queue, refreshed every 15 s. **Scenario:** before the morning stand-up the ops head sees Delhi and Nagpur under thunderstorm (real Open-Meteo data), two critical shipments on those corridors, and approves a re-ranking recommendation that saves ₹4,200 — one click, audited.

Shipments & AI assignment — “which truck takes this load?”

How it works: creating a shipment resolves the real road distance, ETA, cost and weather risk before it is saved. The suggestion panel ranks available vehicles with score breakdowns (capacity fit, proximity, health, driver hours). Assignment and status advances follow a strict state machine (draft → scheduled → assigned → in transit → delivered/exception) with a full event log. **Scenario:** an 8.2 t steel-coil job from Mumbai to Nagpur is offered three trucks; the dispatcher assigns the top-scoring one (score 72.3) in one click and the truck, driver and timeline update instantly.

Route Intelligence — planning around reality

How it works: pick origin, destination, weight and cargo; the optimizer returns Fastest / Balanced / Economy alternatives using real BRouter/OSRM road legs, live weather, fuel, tolls, CO₂ and risk per option, with every factor explained. **Scenario:** Mumbai → Delhi with a thunderstorm over Rajasthan: Economy via Nashik (NH-43) wins on cost with the lowest emission while avoiding the worst weather cell.

Consolidation — filling half-empty trucks

How it works: threshold-driven grouping finds unassigned loads that share a corridor and departure window and fit one vehicle (weight + volume fill bars shown); applying a group creates a consolidated load with computed savings. **Scenario:** three Mumbai → Nagpur part-loads (totalling 20 t against 28 t capacity) merge into one trip and save ≈ ₹42,800 — money that would have burned as diesel on two extra trucks.

Return Loads — no truck runs empty

How it works: vehicles nearing delivery are matched against open loads near their destination; each card shows detour km, extra profit and a factor-explained match score. **Scenario:** a truck finishing in Kanpur picks up a 9 t load back to Delhi for +₹18,400 profit on a 61 km detour — the difference between a profitable round trip and a dead-leg return.

Disruption Control — the playbook button

How it works: incidents carry severity, affected-shipment counts and a step-by-step playbook (re-route → notify client → escalate → resolve). Running a step re-scores and re-routes the real affected shipments and tells you exactly which refs were touched. **Scenario:** a flood closes Mumbai–Pune; the dispatcher runs the playbook, three shipments are re-ranked via alternate corridors and each affected ref is listed in the toast.

Analytics, Copilot & Thresholds

Analytics: 7/30/90-day trends (on-time %, ₹/km, CO₂, utilisation), top corridors, driver leaderboard and status mix. **Copilot:** ask “which corridors have weather risk right now?” and it answers with the real Open-Meteo numbers for your corridors, labelled LLM + your data (or rule_based if the LLM is offline). **AI Thresholds:** every weight and band behind the engines (45 parameters, 14 groups) is editable by admins; saving bumps the config version (cfg vN shown across the UI) and engines re-score live — no redeploy.

7 • API Reference (31 endpoints)

All endpoints live under `/api/*` on the FastAPI service (interactive docs: **/docs**). Authentication is a session cookie; admin-only routes are marked †.

Group	Endpoints
Auth	POST /auth/register · POST /auth/login · POST /auth/logout · GET /auth/me
Command center	GET /overview — KPIs, risk feed, recommendations, map payload, live weather, sources
Shipments	GET/POST /shipments · GET /shipments/{id} · GET /shipments/{id}/suggestions · POST /shipments/{id}/assign · POST /shipments/{id}/advance
Fleet	GET /fleet — vehicles, drivers, utilisation summary, maintenance flags
Analytics	GET /analytics — 7/30/90-day trends (on-time %, cost/km, CO ₂ , utilisation), top corridors, driver leaderboard, status mix
Route optimization	POST /routes/optimize — 3 alternatives with live road + weather meta
Consolidation	GET /consolidation · POST /consolidation/apply
Return loads	GET /return-loads · POST /return-loads/accept
Disruptions	GET /disruptions · POST /disruptions/{id}/action (run_playbook / escalate / resolve)
Recommendations	POST /recommendations/{id}/decide (approve / reject, audited)
Notifications	GET /notifications · POST /notifications/read
Thresholds	GET /thresholds · PUT /thresholds † · POST /thresholds/reset †
Copilot & misc	POST /copilot · GET /live/conditions (public) · POST /demo-request (public) · GET /health · GET /api (index)

8 • Security & Data Honesty

The backend is built multi-tenant from day one: every query is filtered by the caller's organisation (org isolation is enforced in the data layer, not the UI), roles gate write operations, and destructive or high-impact actions are audited with notifications.

- **Passwords:** scrypt (N=16384, 32-byte key) with per-user salt; timing-safe comparison.
- **Sessions:** opaque 32-byte tokens in HttpOnly SameSite cookies, 7-day TTL, server-side revocation on logout.
- **RBAC:** ANALYST is read-only; threshold changes require ADMIN and bump the config version with an audit trail.
- **Org isolation:** shipments, fleet, recommendations and notifications are scoped per organisation — verified with a two-tenant test suite.
- **Data honesty:** the live / rule_based / optimization / demo labels flow from the API meta into badges on every screen, so judges and users always know what they are looking at.

9 • Running the Project

Prerequisites: Node/Bun for the frontend, Python 3.12+ for the backend. From the repository root:

```
# 1 • Backend (FastAPI, port 8000)
cd backend
pip install -r requirements.txt
python -m uvicorn app.main:app --port 8000    # auto-creates + seeds the DB

# 2 • Frontend (Next.js, port 3000) – in a second terminal
cd frontend
npm install && npm run dev                    # proxies /api/* to :8000

# Open http://localhost:3000 – backend docs at http://localhost:8000/docs
```

The database file self-seeds on first boot (2 organisations, 6 users, 30+ vehicles, 25+ drivers, 160+ shipments, disruptions, recommendations). Demo credentials: **admin@meridian.in** · **ops@meridian.in** · **analyst@meridian.in** — password **Demo@12345** (analyst is read-only; admin@kalinga.in proves org isolation). Docker Compose and a GitHub Actions CI workflow are included; `AUTO_START_PY_BACKEND=0` disables the dev auto-spawn if you run uvicorn separately.

10 · SIH Innovation Register & Honest Limitations

What is genuinely innovative: explainable AI everywhere (factor bars behind every score), threshold governance with instant propagation (cfg vN chips), live public-data integration with attribution and graceful labelled fallbacks, corridor playbooks that touch real shipments, and an honest data-status contract that refuses to fake telemetry.

- Limitation (by design): fleet telemetry is a labelled demo dataset — swapping in GPS/GPS provider feeds only requires new writers to the same Vehicle/Shipment tables.
- SQLite keeps the demo zero-dependency; the SQLAlchemy models are PostgreSQL-ready.
- Public routers (BRouter/OSRM) are rate-limited by design of the caching layer; production can point at a self-hosted OSRM via env without code changes.
- The copilot is scoped to your organisation's data — it cannot browse the web and will say so rather than improvise.

Verification note. This documentation describes the verified system as shipped: 31 FastAPI endpoints oracle-diffed against the previous contract, all core flows exercised end-to-end in a real browser, live weather and road data confirmed on-screen with source labels.